Max Path Sum In Binary Tree

AlgoExpert · Hard · Binary Trees

The problem

- Maximum sum over any path (node-to-node, no repeats).

Intuition

- At each node, the best path either bends through it or continues upward through one child.

Brute force vs optimal

- Brute force: every pair of nodes as endpoints → $O(n^2)$
- Post-order: return best single branch, update best bend → $O(n)$

Approach

- 1. Post-order returns the best downward branch
- 2. A bending path is node plus both positive branches
- 3. Track the global max

Worked example




```
node 4: through = 4, returns 4
best = 4
```

Node 4: path through = 4

Worked example




```
node 5: through = 5, returns 5
best = 5
```

Node 5: path through = 5

Worked example




```
node 2: through = 11, returns 7
best = 11
```

Node 2: path through = 11

Worked example




```
node 7: through = 7, returns 7
best = 11
```

Node 7: path through = 7

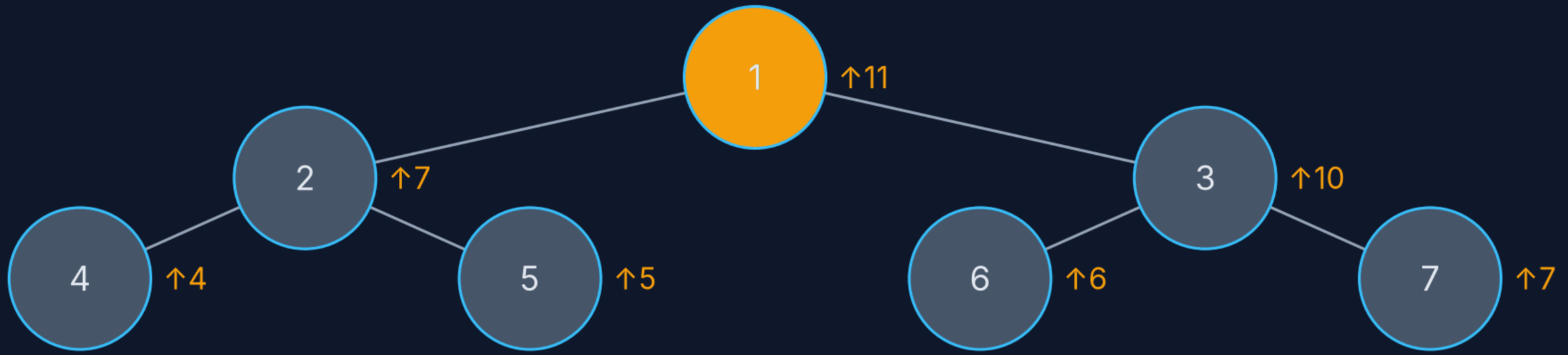
Worked example




```
node 3: through = 16, returns 10
best = 16
```

Node 3: path through = 16

Worked example




```
node 1: through = 18, returns 11
best = 18
```

Node 1: path through = 18

Java solution

```java
public class MaxPathSumInBinaryTree {
    public static int maxPathSum(BinaryTree t) { int[] best = {Integer.MIN_VALUE}; branch(t, best); return best[0]; }

    private static int branch(BinaryTree t, int[] best) {
        if (t == null) return 0;
        int l = Math.max(0, branch(t.left, best)), r = Math.max(0, branch(t.right, best));
        best[0] = Math.max(best[0], t.value + l + r);
        return t.value + Math.max(l, r);
    }
}
```

Complexity

- Time: O(n)
- Space: O(h)

Edge cases & pitfalls

- Clamp negative branches to 0
- All-negative tree: answer is the max node
- Return a single branch, not both

Interview tips & follow-ups

- Explain 'return branch vs record bend'

Recap

- At each node, the best path either bends through it or continues upward through one child.
- Complexity: O(n)